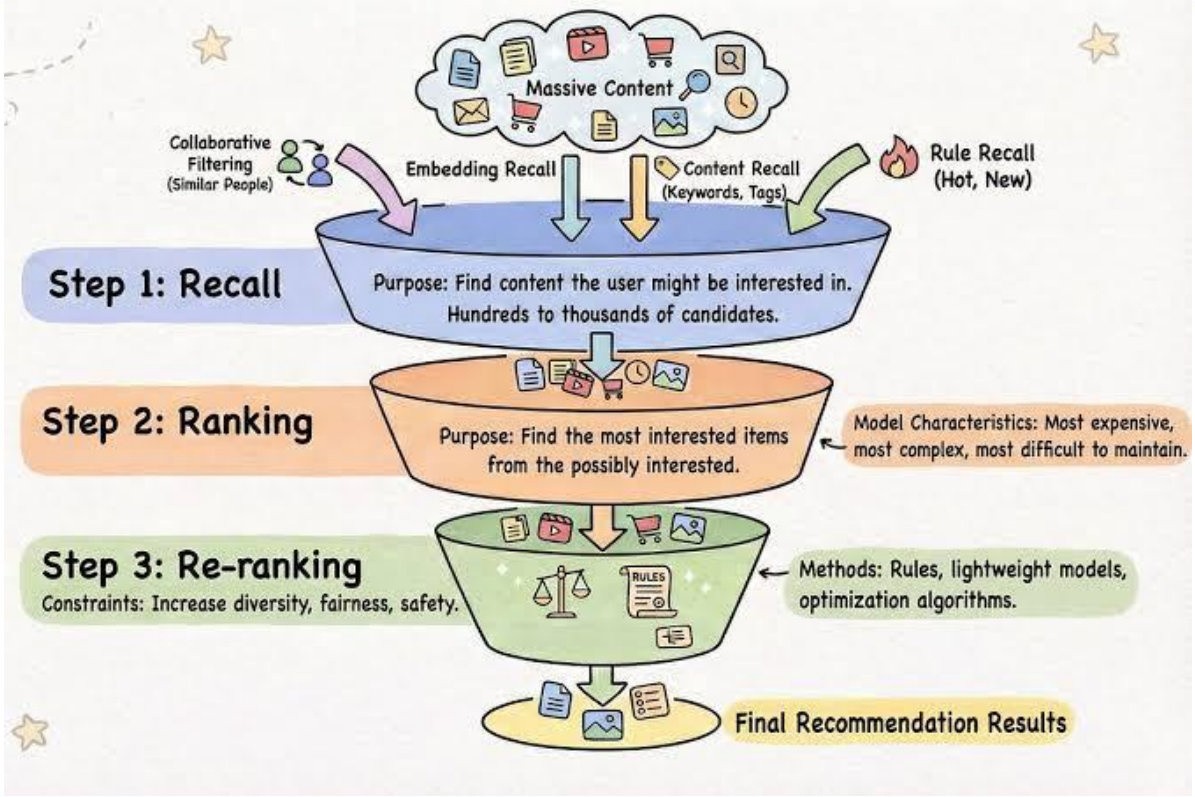
ITMO

Ranking and Re-Ranking

Еще раз о воронке



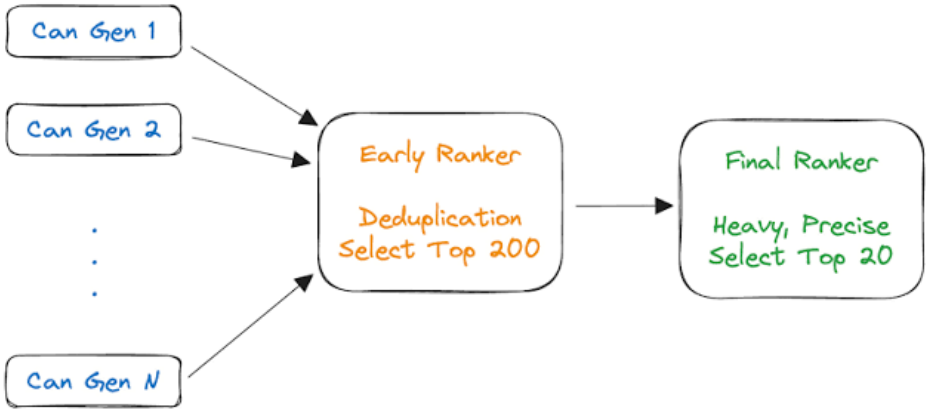
ITMO

Ranking

Мотивация

1. Хотим уметь объединять кандидатов от разных кандгенов
2. Хотим повысить общее качество ранжирования

Ранкинг дороже кандидатогенерации, это тяжеловесные модели с Early Fusion парадигмой



Данные в ранжировании

Табличный формат: (group_id, item_id, [features], target_label)

Целевая переменная может быть бинарной (0/1) или небинарной (e.g. 1-5), но она должна позволять отранжировать item_id при группировке по group_id

group_id = session_id / request_id / date+user_id

Query / group	User features			Item features			Context features		Aggregated features		Cross feature	Target
request_id	user_id	user_age	user_country	item_id	item_category	item_price	device	time_of_day	user_clicks_7d	item_ctr_7d	user_cat_clicks_30d	clicked
req_101	u42	25	US	i10	electronics	120	mobile	morning	15	0.09	6	1
req_101	u42	25	US	i11	books	20	mobile	morning	15	0.04	1	0
req_101	u42	25	US	i12	clothing	65	mobile	morning	15	0.07	3	0
req_102	u87	32	DE	i13	books	18	desktop	evening	4	0.12	8	1
req_102	u87	32	DE	i14	electronics	230	desktop	evening	4	0.06	0	0
req_102	u87	32	DE	i15	clothing	50	desktop	evening	4	0.08	2	0

Feature Engineering

Агрегаты и кросс-счетчики считаются в рамках определенного окна

Raw features (original data)

User features

Categorical	user_id, country, language, device_type, ...
Numerical	age, account_age, #sessions, ...
Sequential / set	recent_items, recent_queries, user_interests, ...
Text / content	profile_text, bio, ...

Item features

Categorical	item_id, category, brand, seller_id, ...
Numerical	price, rating, popularity, inventory, ...
Sequential / set	item_tags, co-viewed items, co-purchased items, ...
Text / content	title, description, image (CNN), ...

Context / interaction features

Categorical	time_of_day, day_of_week, location, traffic_source, ...
Numerical	time_since_last_click, position, session_length, ...
Sequential / set	previous_items, current_session_items, query_sequence, ...
Other	is_new_user, interaction_type (click, view, ...), ab_test_group, ...

Derived features (built from raw features)

Aggregated features

Statistics over user/item/history, windows, counts, rates, recency, etc.

User aggregates	#items_clicked, #purchases, avg_rating, ctr, days_since_last_purchase, unique_categories, ...
Item aggregates	#views, #purchases, avg_rating, conversion_rate, days_since_published, #unique_users, ...
Interaction / session aggregates	user-item click count, time since last interaction, session click count, avg dwell time, bounce rate, ...

Cross / combinational features

Explicit feature interactions between different features

User × item	user_id × item_id, user_country × item_category, ...
User × context	user_segment × time_of_day, device_type × item_category, ...
Item × context	item_category × time_of_day, item_price × position, ...
Multiple features	user_interests × item_tags, is_new_user × position, user_country × item_category × time_of_week, ...

Метрики

$$NDCG@K = \frac{DCG@K}{IDCG@K}$$

$$DCG@K = \sum_{i=1}^{K} \frac{G(y_i)}{\log_2(i+1)}$$

Linear NDCG: $G(y) = y$

Exponential NDCG: $G(y) = 2^y - 1$

$$AUC = \frac{1}{|P||N|} \sum_{p \in P} \sum_{n \in N} \left[\mathbb{I}(s_p > s_n) + \frac{1}{2} \mathbb{I}(s_p = s_n) \right]$$

$$GAUC = \frac{\sum_{g=1}^{G} w_g \cdot AUC_g}{\sum_{g=1}^{G} w_g}$$

$$PairAUC = \frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} \left[\mathbb{I}(s_i > s_j) + \frac{1}{2} \mathbb{I}(s_i = s_j) \right]$$

Функции потерь

Pointwise, Pairwise, Listwise ->

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \sigma(s_i) + (1-y_i)\log(1-\sigma(s_i))\right]$$

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(y_i - s_i)^2$$

$$\mathcal{L}_{\text{ListNet}} = -\sum_{i=1}^{n}\frac{e^{y_i}}{\sum_{j=1}^{n} e^{y_j}}\log\left(\frac{e^{s_i}}{\sum_{j=1}^{n} e^{s_j}}\right)$$

$$\mathcal{L}_{\text{BPR}} = -\frac{1}{|\mathcal{P}|}\sum_{(i,j)\in\mathcal{P}}\log\sigma(s_i - s_j)$$

$$\mathcal{L}_{\text{ListMLE}} = -\sum_{k=1}^{n}\log\frac{e^{s_{\pi_k^*}}}{\sum_{j=k}^{n} e^{s_{\pi_j^*}}}$$

А может получится как-то оптимизировать NDCG напрямую?..

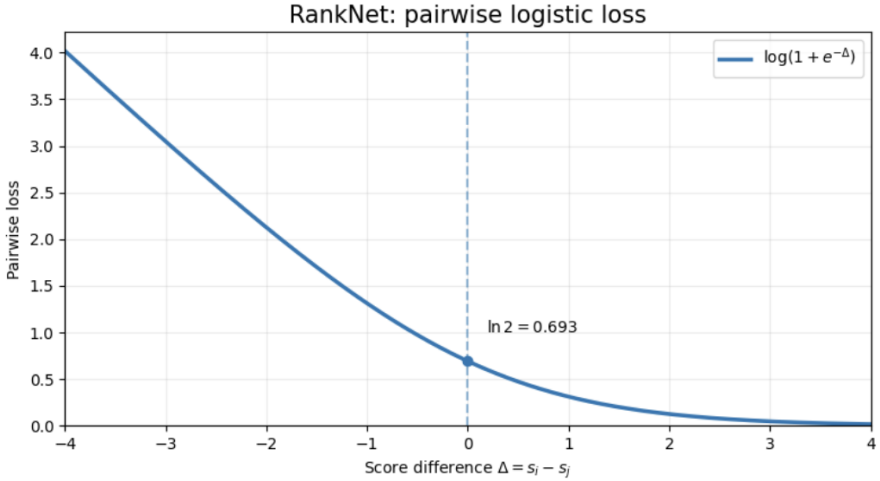
RankNet

Модель учится отвечать на вопрос: какой из двух объектов должен находиться выше?

$$P(i \succ j) = \sigma(s_i - s_j) = \frac{1}{1 + e^{-(s_i - s_j)}}$$

$$\mathcal{L}_{\text{RankNet}} = \frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} \log\left(1 + e^{-(s_i - s_j)}\right)$$

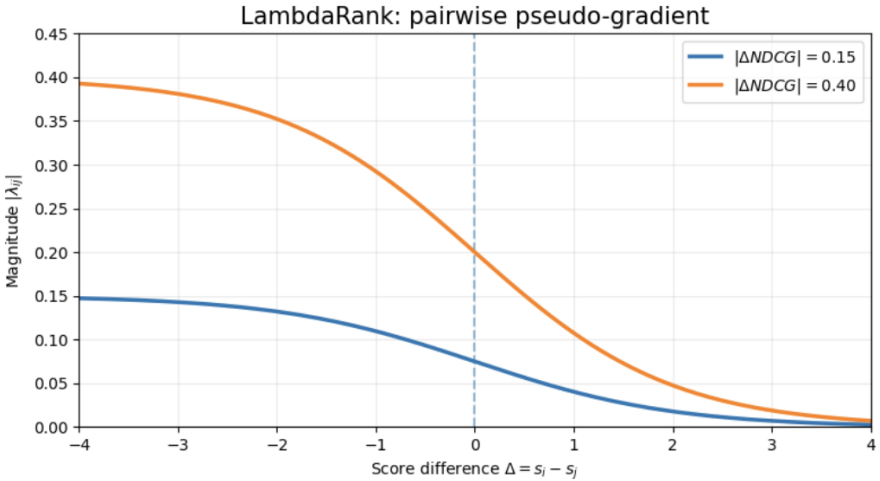
Не учитывает положение объектов в выдаче. Ошибка между позициями 1 и 2 может штрафоваться точно так же, как ошибка между позициями 99 и 100, если разницы скоров одинаковые



LambdaRank

LambdaRank учитывает, насколько изменится NDCG, если поменять два объекта местами

$$|\Delta NDCG_{ij}| = \frac{|G(y_i) - G(y_j)|}{IDCG} \left| \frac{1}{\log_2(r_i + 1)} - \frac{1}{\log_2(r_j + 1)} \right|$$



$$\lambda_{ij} = -\frac{|\Delta NDCG_{ij}|}{1 + e^{s_i - s_j}}, \qquad y_i > y_j$$

$$\lambda_i = \sum_{j:y_i > y_j} \lambda_{ij} - \sum_{j:y_j > y_i} \lambda_{ji}$$

$$\tilde{\mathcal{L}}_{\text{LambdaRank}} = \sum_{(i,j) \in \mathcal{P}} w_{ij} \log(1 + e^{-(s_i - s_j)})$$

Обновляем итеративно при изменении ранжирования

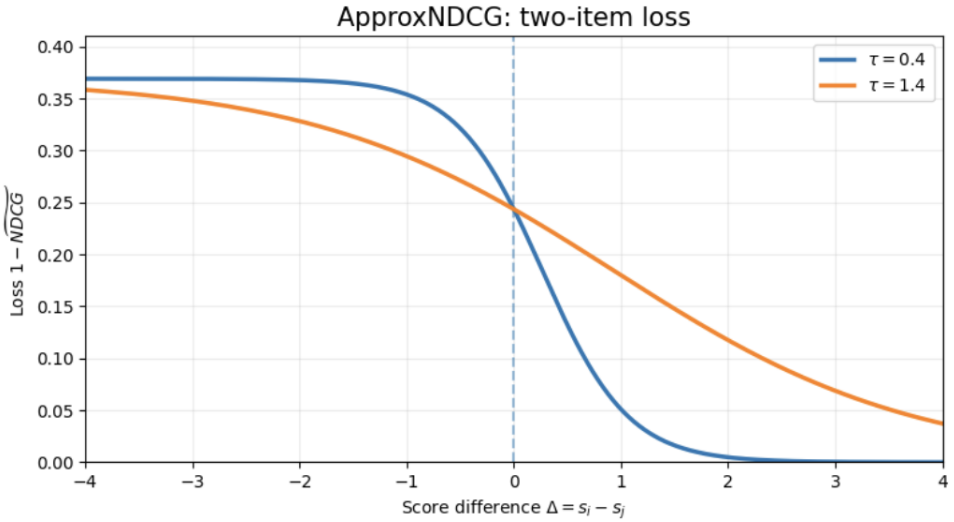
ApproxNDCG

Аппроксимируем дискретный порядок с помощью гладкой сигмоиды

$$r_i = 1 + \sum_{j \neq i} \mathbb{I}(s_j > s_i)$$

$$\tilde{r}_i = 1 + \sum_{j \neq i} \frac{1}{1 + e^{-(s_j - s_i)/\tau}}$$

$$\mathcal{L}_{\text{ApproxNDCG}} = 1 - \frac{1}{IDCG} \sum_{i=1}^{n} \frac{2^{y_i} - 1}{\log_2 \left(2 + \sum_{j \neq i} \frac{1}{1 + e^{-(s_j - s_i)/\tau}}\right)}$$



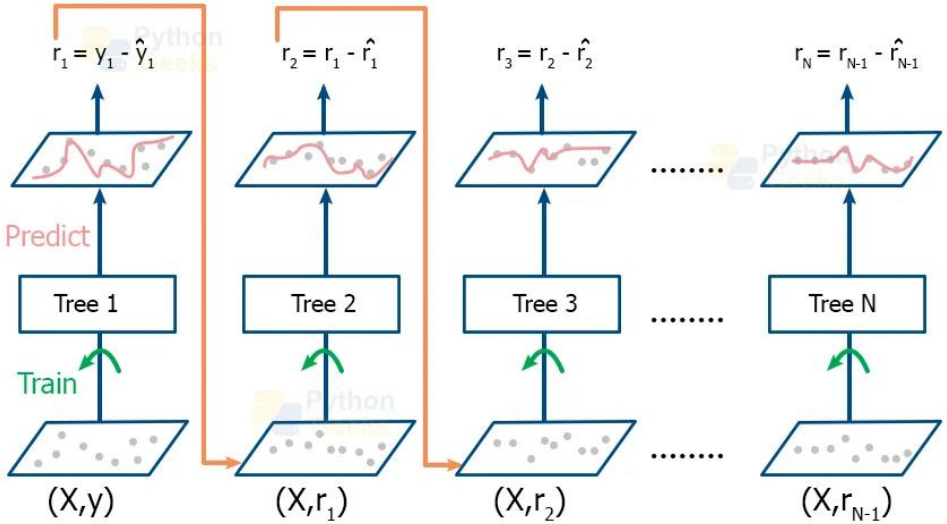
GBDT

Градиентный спуск в пространстве функций

$$r_i = -\frac{\partial L(y_i, F_{t-1}(x_i))}{\partial F_{t-1}(x_i)}$$

$$h_t(x_i) \approx r_i$$

$$F_t(x) = F_{t-1}(x) + \eta h_t(x)$$

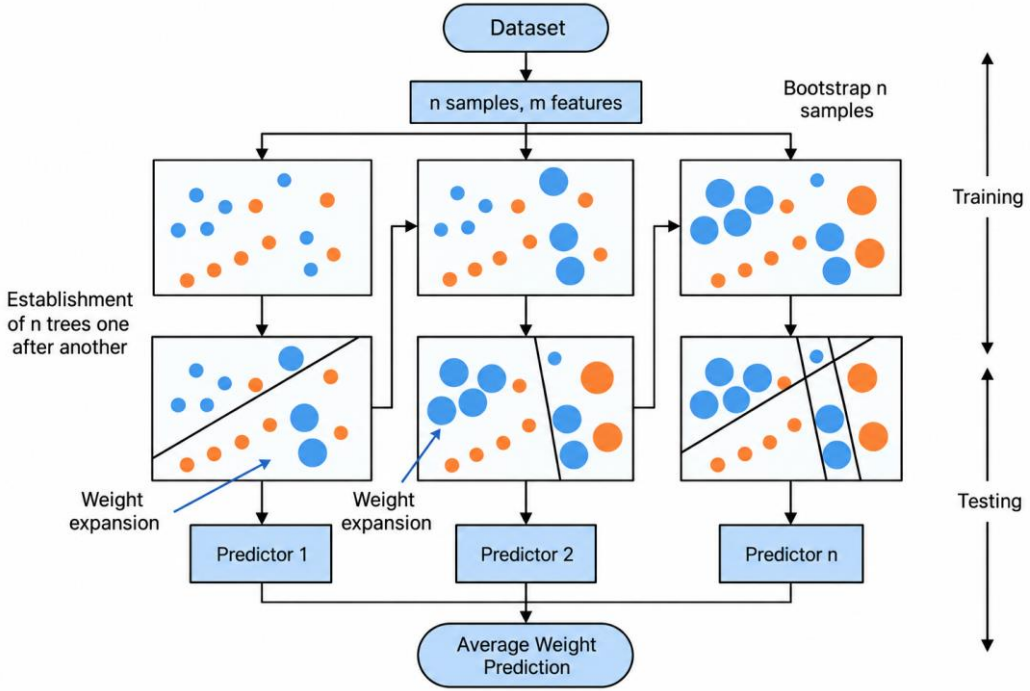


CatBoost

Библиотека от Яндекса, нативно поддерживает категориальные фичи (не нужно готовить их вручную)

- CatBoostRegressor
- CatBoostClassifier
- CatBoostRanker (LambdaRank, YetiRank)

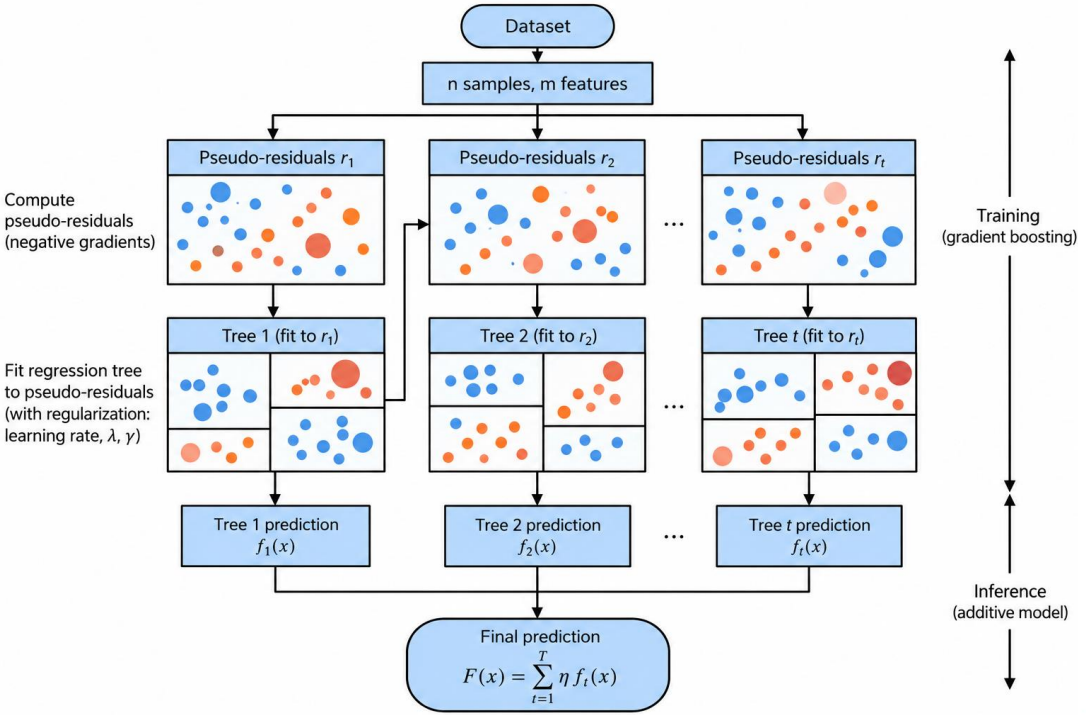
Внимательно читайте документацию!



XGBoost

Еще одна популярная библиотека

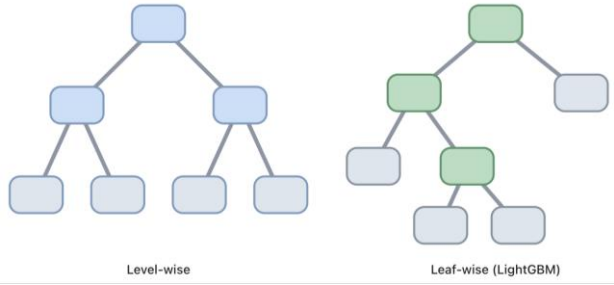


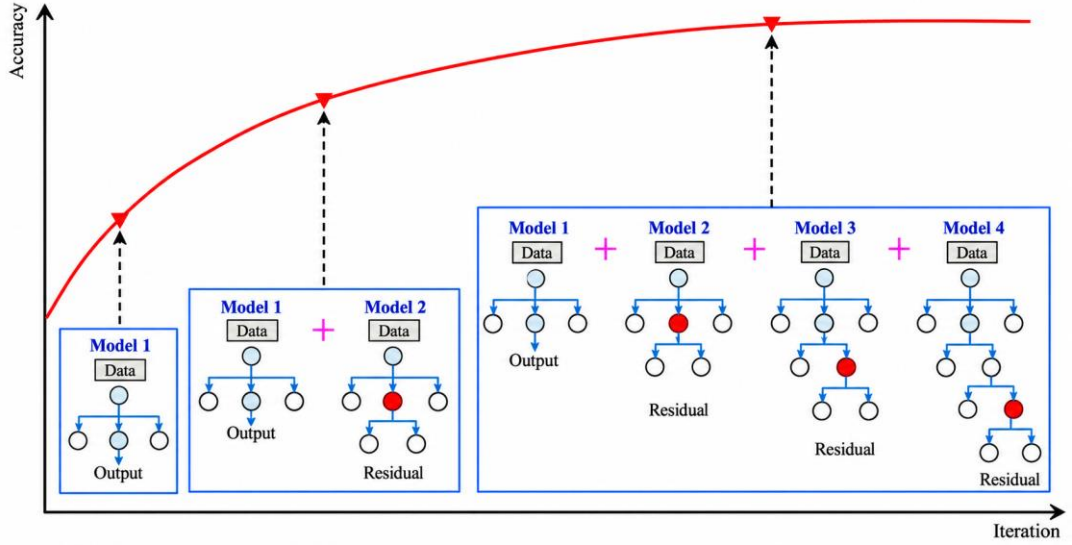


LightGBM

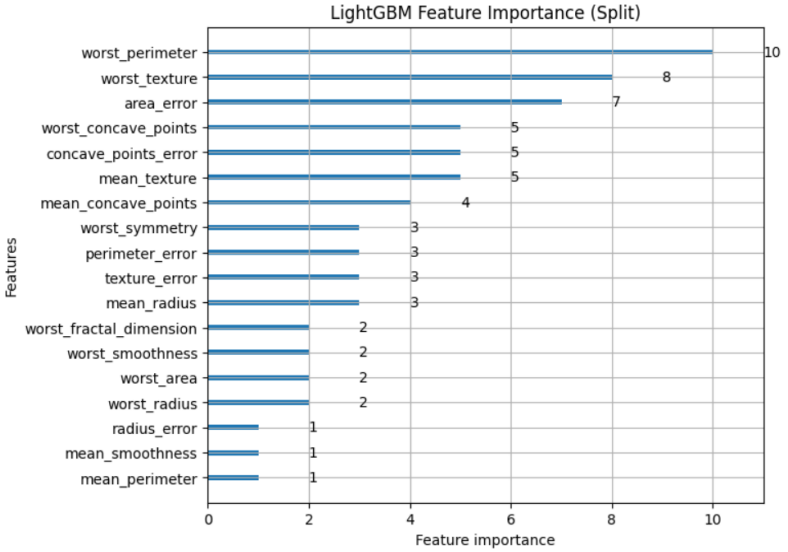
Тоже градиентный бустинг, но с оптимизированным построением деревьев

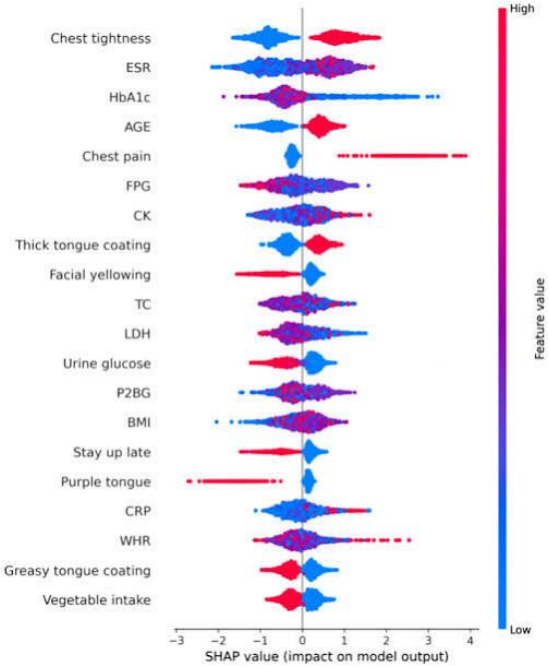
$$g_i = \frac{\partial L}{\partial F(x_i)}, \qquad H_i = \frac{\partial^2 L}{\partial F(x_i)^2}$$



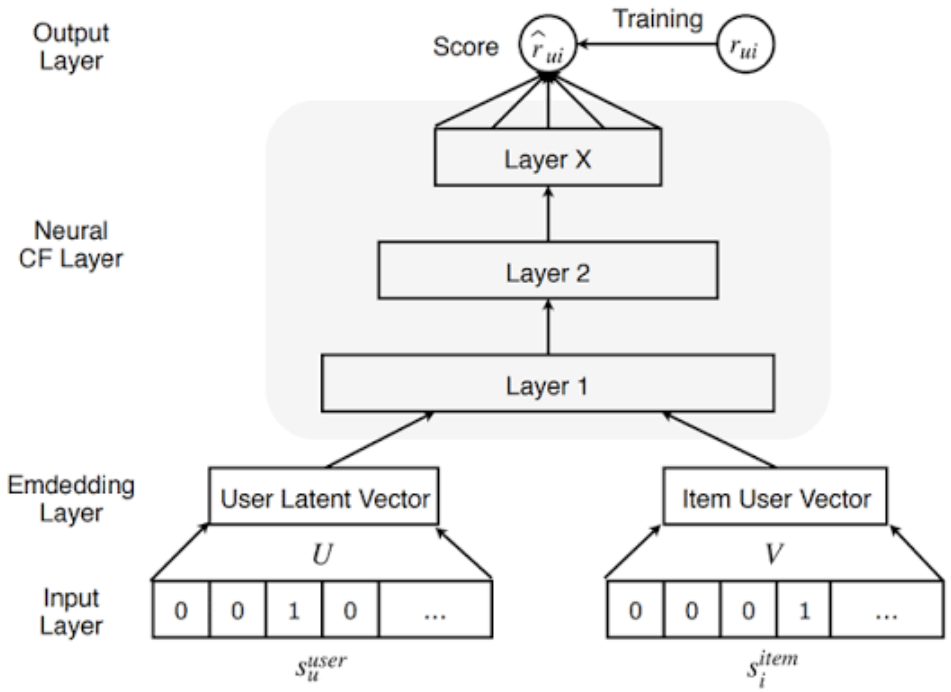


Feature Importance





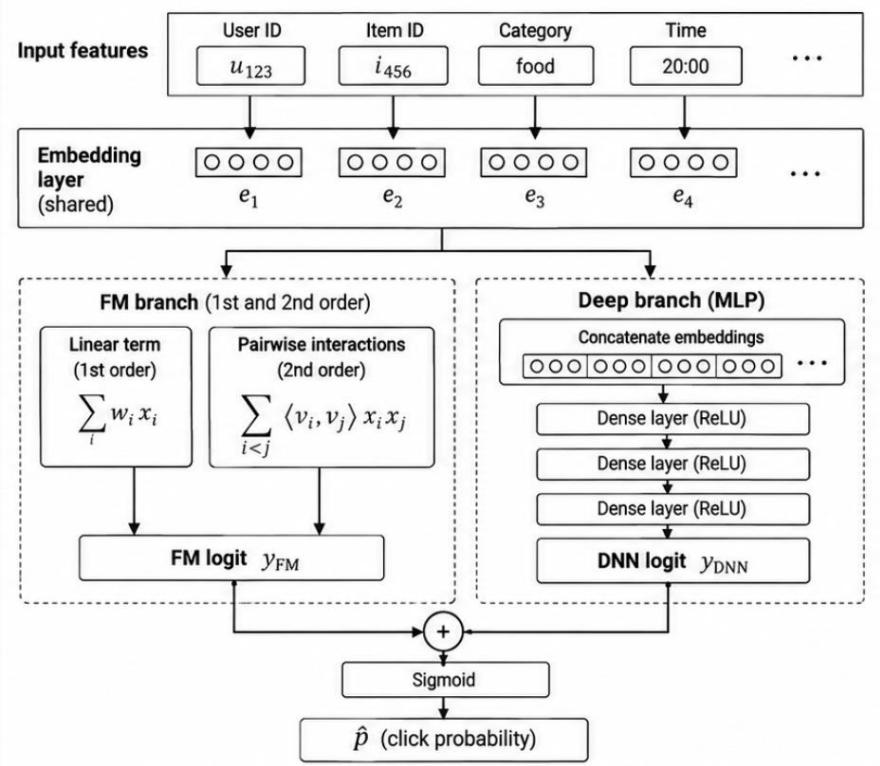
Neural Ranking



DeepFM

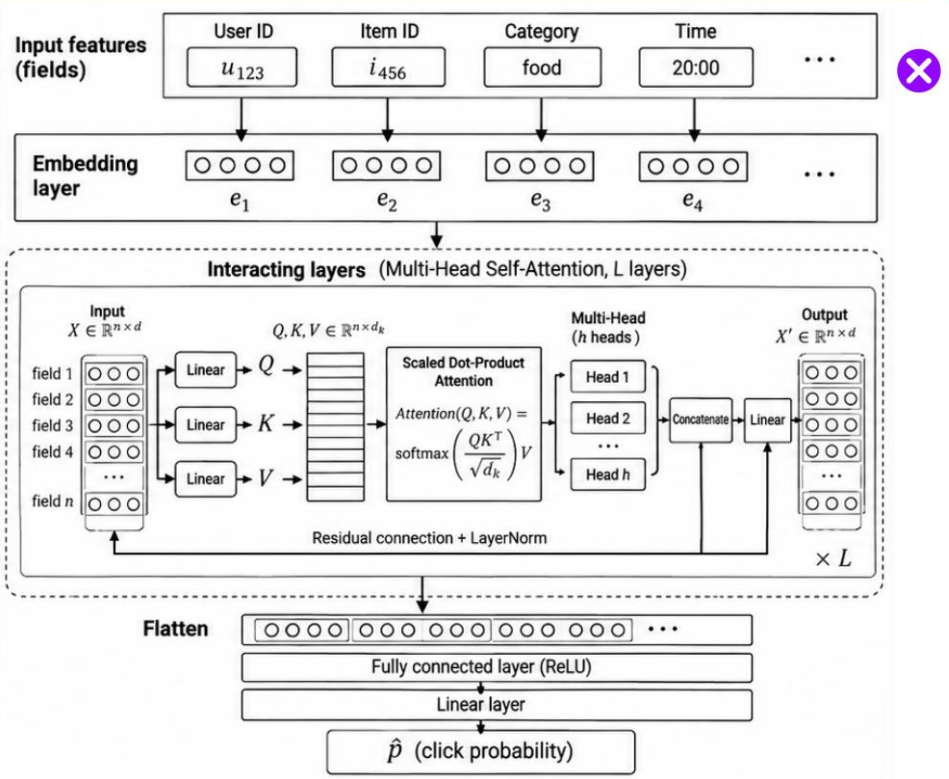
Идея в том, чтобы объединить две модели, которые хорошо работают с разными типами взаимодействий:

- Factorization Machine (FM) моделируют линейные эффекты и попарные взаимодействия между признаками через скалярное произведение эмбеддингов
- Deep Neural Network (MLP) неявно моделируют более сложные нелинейные зависимости, включая взаимодействия высокого порядка

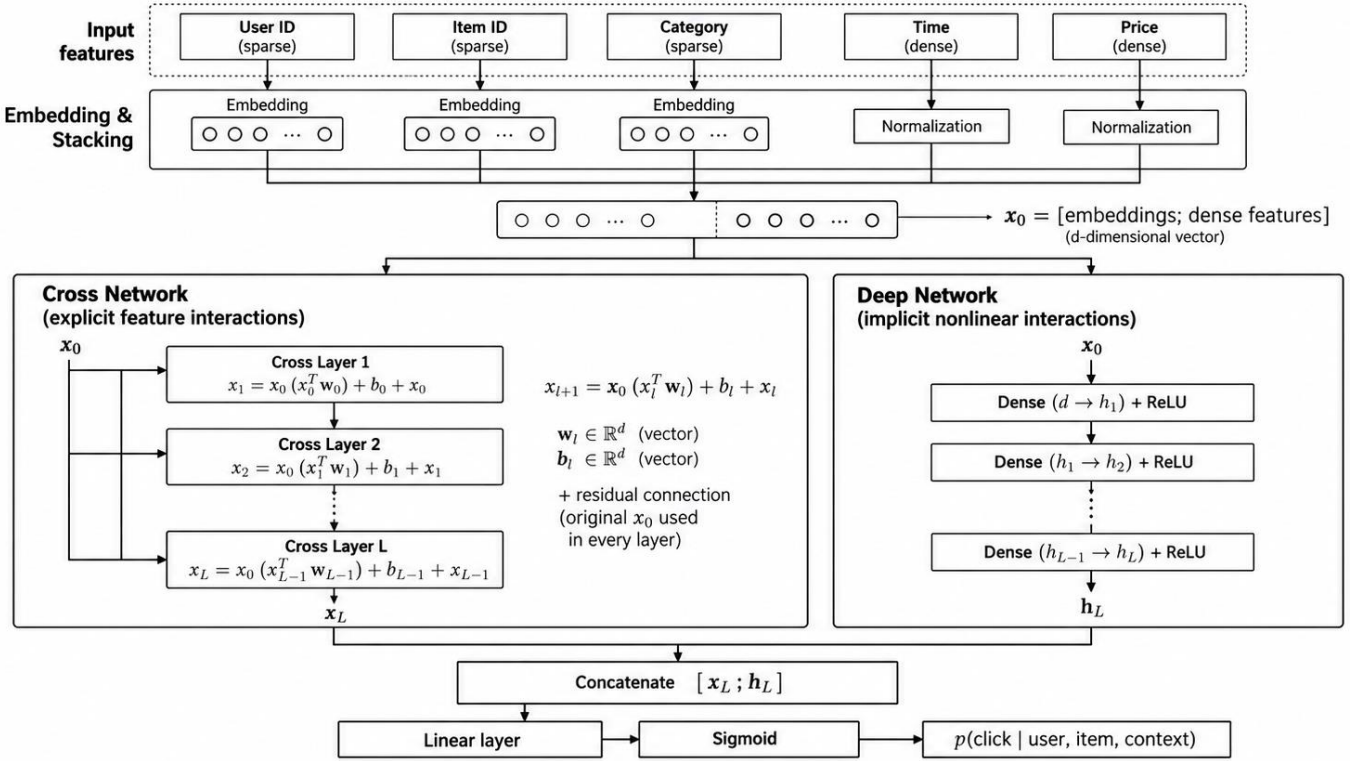


AutoInt

Энкодим фичи, а дальше накидываем многоголовое внимание, чтобы каждая из фичей могла смотреть на все остальные



DCN


Input features
User ID (sparse)
Item ID (sparse)
Category (sparse)
Time (dense)
Price (dense)
Embedding & Stacking
Embedding
Embedding
Embedding
Normalization
Normalization
x_0 = [embeddings; dense features]
(d-dimensional vector)
Cross Network
(explicit feature interactions)
x_0
Cross Layer 1
$x_1 = x_0 (x_0^T w_0) + b_0 + x_0$
Cross Layer 2
$x_2 = x_0 (x_1^T w_1) + b_1 + x_1$
Cross Layer L
$x_L = x_0 (x_{L-1}^T w_{L-1}) + b_{L-1} + x_{L-1}$
x_L
$x_{l+1} = x_0 (x_l^T w_l) + b_l + x_l$
$w_l \in \mathbb{R}^d$ (vector)
$b_l \in \mathbb{R}^d$ (vector)
+ residual connection
(original x_0 used
in every layer)
Deep Network
(implicit nonlinear interactions)
x_0
Dense $(d \to h_1)$ + ReLU
Dense $(h_1 \to h_2)$ + ReLU
Dense $(h_{L-1} \to h_L)$ + ReLU
h_L
Concatenate $[x_L ; h_L]$
Linear layer
Sigmoid
p(click | user, item, context)

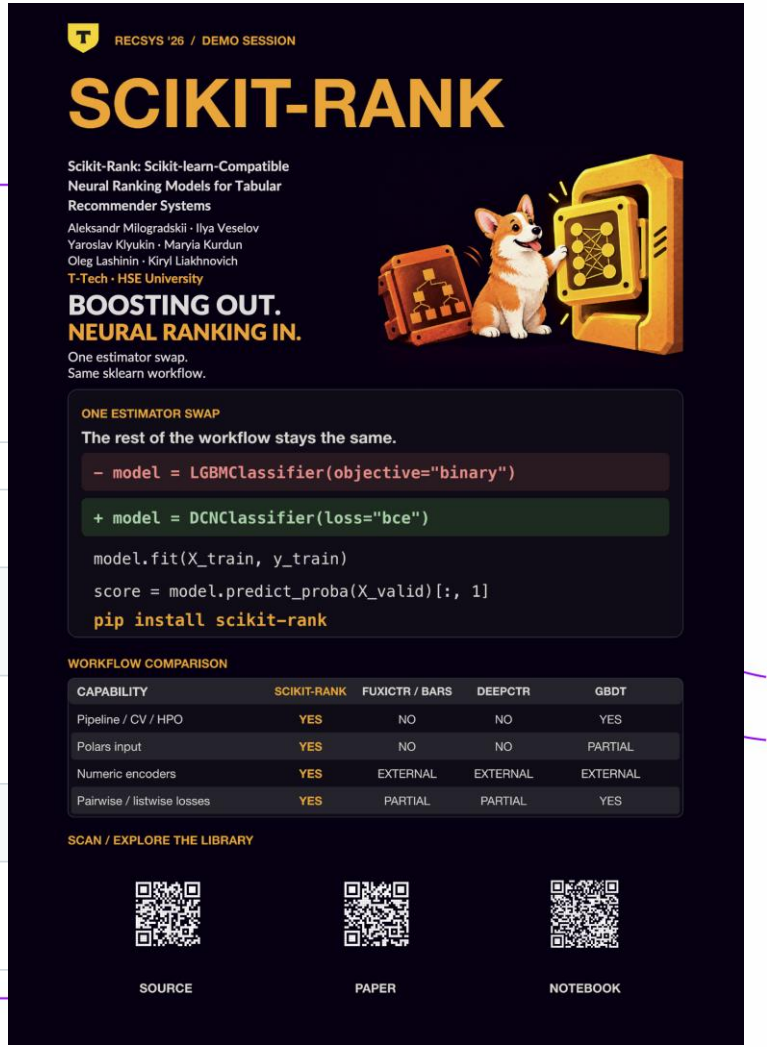
Scikit-Rank

sklearn-compatible neural estimators for tabular ranking, classification, and regression.

`scikit-rank` provides neural, drop-in alternatives to LightGBM / CatBoost / XGBoost for tabular problems. Its model zoo pairs modern tabular backbones with a consistent scikit-learn `fit` / `predict` / `predict_proba` API, so estimators work in existing sklearn pipelines and with `GridSearchCV`, `clone`, `get_params`, and `set_params`.

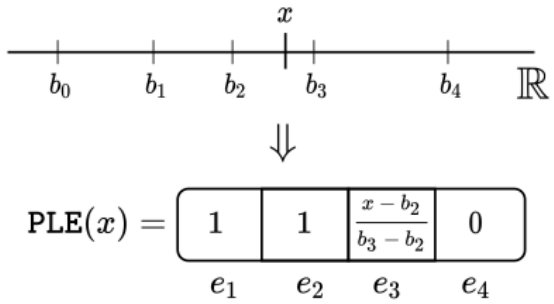
```
from scikit_rank import DCNClassifier, FinalNetClassifier, TabMClassifier
```

Backbone	sklearn estimators	Architecture options
DCNv2	`DCNClassifier`, `DCNRegressor`, `DCNRanker`	`hidden_units`, `cross_layers`, `cross_rank`, `structure`, `cross_type`
FinalNet	`FinalNetClassifier`, `FinalNetRegressor`, `FinalNetRanker`	`block_type`, `block1_hidden_units`, `block2_hidden_units`, `use_field_gate`, `use_2b_consistency_loss`
FinalMLP	`FinalMLPClassifier`, `FinalMLPRegressor`, `FinalMLPRanker`	`mlp1_hidden_units`, `mlp2_hidden_units`, `use_fs`, `fs_hidden_units`, `fs1_context`, `fs2_context`, `num_heads`
TabM	`TabMClassifier`, `TabMRegressor`, `TabMRanker`	`arch_type`, `n_blocks`, `d_block`, `dropout`, `k`, `start_scaling_init`
DESTINE	`DESTINEClassifier`, `DESTINERegressor`, `DESTINERanker`	`embedding_dim`, `attention_dim`, `num_heads`, `attention_layers`, `unary_mode`, `residual_mode`, `use_wide`



Кодирование признаков

- Categorical – One-Hot / Multi-Hot
- Numeric – PLE (Piecewise Linear Encoding)
- Dense – Linear Projection
- Temporal – Bucketing



$$b(\Delta t) = \min\left(B - 1, \left\lfloor \log_2\left(1 + \frac{\Delta t}{\tau}\right)\right\rfloor\right)$$

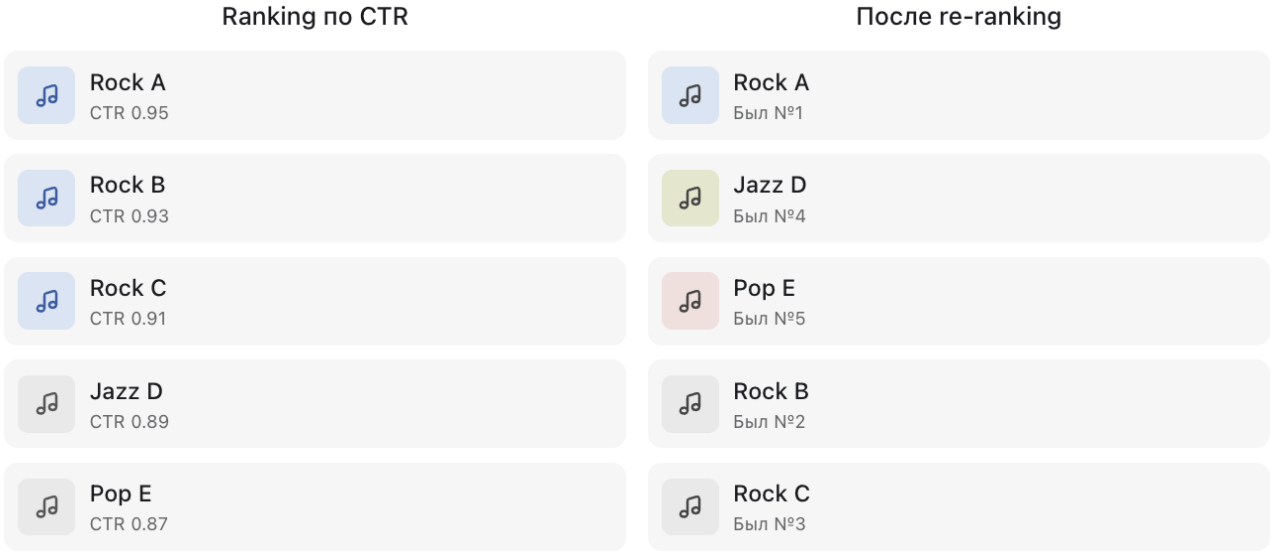
Объединяем за счет пулинга: concat, sum, weighted sum

ITMO

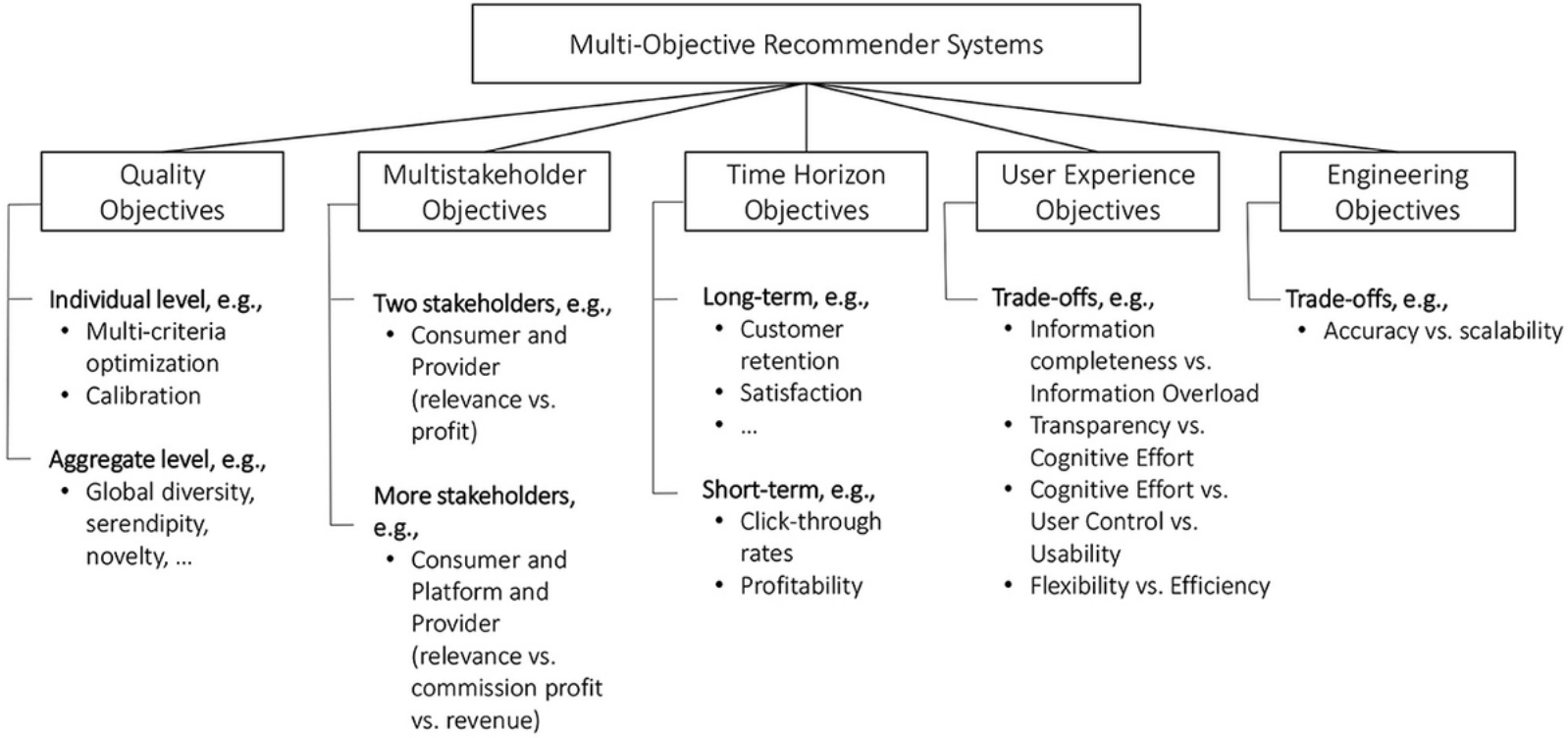
Re-Ranking

Мотивация

Финальное упорядочивание выдачи, чтобы она была не только релевантна, но и приятна визуально



Multi-objective Recommendations



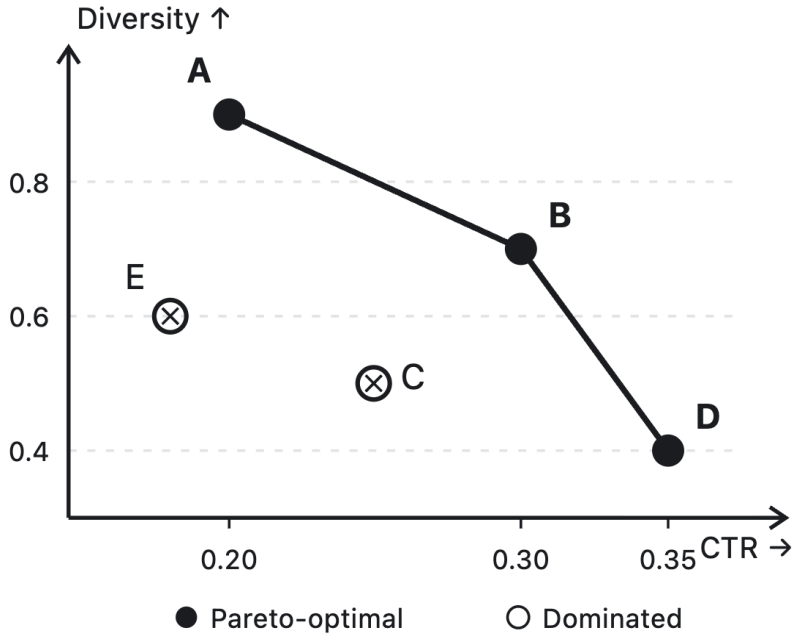
Парето-оптимальность

Не существует одного универсально лучшего ранжирования – существует множество оптимальных компромиссов между целями

$$\pi_a \succ_P \pi_b \iff \begin{cases} f_j(\pi_a) \geq f_j(\pi_b), & \forall j, \\ \exists k: \ f_k(\pi_a) > f_k(\pi_b). \end{cases}$$

$$\pi^* \in \Pi_{\text{Pareto}} \iff \nexists \pi \in \Pi : \pi \succ_P \pi^*$$

$$\mathcal{F}_{\text{Pareto}} = \{\mathbf{f}(\pi) \mid \pi \in \Pi_{\text{Pareto}}\}$$



Блендинг

Блендинг – это процесс смешивания выдачи разных моделей

Можно объединять, взвешивая скоры с определенными бизнесом коэффициентами, если они находятся в одном диапазоне

$$S(u,i) = \sum_{m=1}^{M} \alpha_m \, \tilde{s}_m(u,i)$$

Если скоры находятся в разных диапазонах – можно опираться на порядок от каждого источника и использовать алгоритм Reciprocal Rank Fusion

$$RRF(i) = \sum_{m=1}^{M} \frac{1}{k + \mathrm{rank}_m(i)}$$

Также можно применять квотирование, если заранее определено, какую долю итоговых рекомендаций должен занимать каждый источник

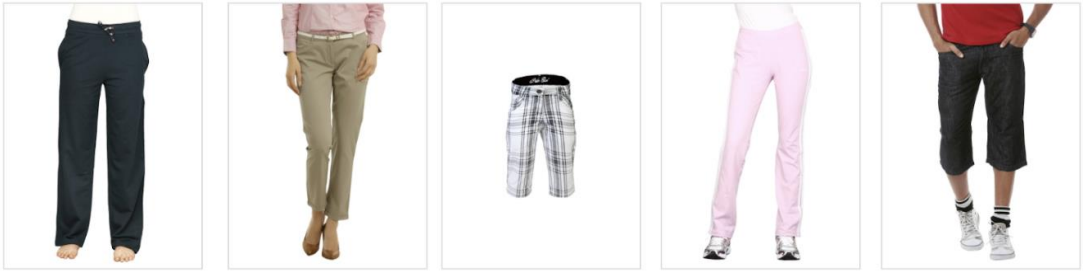
Разнообразие. MMR

На каждом шаге выбираем item, который одновременно релевантен пользователю и не слишком похож на уже выбранные

Before MMR (pure relevance):

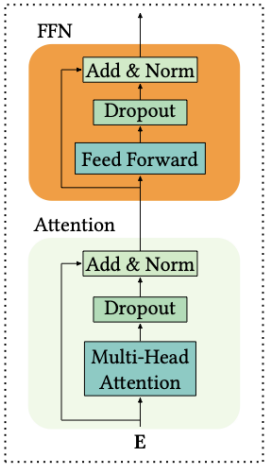


After MMR (λ=0.5):

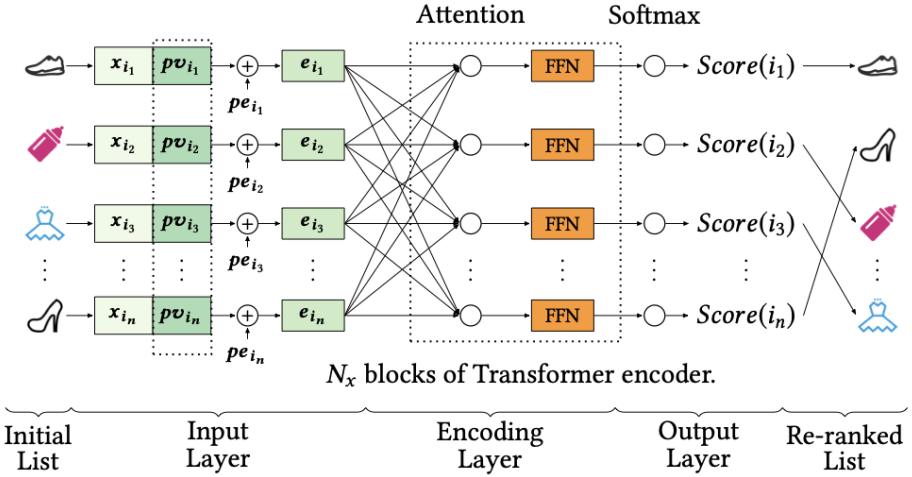


https://www.elastic.co/search-labs/blog/maximum-marginal-relevance-diversify-results

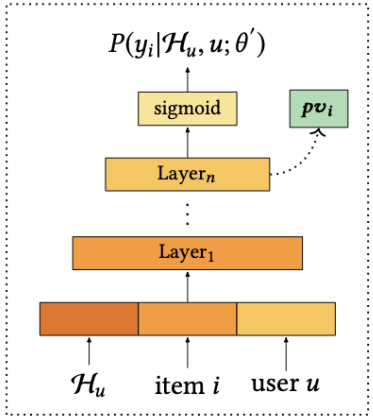
Нейросети. PRM



(a) One block of Transformer encoder.



(b) Architecture of PRM.



(c) The pre-trained model to generate $\boldsymbol{pv}_i$, $i = i_1, \ldots, i_n$.

Personalized Re-ranking for Recommendation

Спасибо за внимание!

ITMO more than a UNIVERSITY

334743@niuitmo.ru